

COURSE DESCRIPTION			
Program	Artificial Intelligence, Artificial Intelligence & Machine Learning, Cloud Computing and Big Data, Cyber Forensics and Information Security, Computer Science and Technology, Computer Science & Engineering, Computer Engineering, Computer Science and Engineering (Artificial Intelligence & Machine Learning), Information Technology, Automation and Robotics, Robotic Process Automation		
Course Title	Problem Solving and Python Programming		
Course Code	1131	Semester	I
Type of Course	Practicum	Contact Hours	90
Teaching Scheme (L: T:P:R)	3:0:3:0	Credits	4.5
CIE Marks	40	ESE Marks	100

Introduction

This foundational Python course emphasizes logical problem-solving and algorithmic thinking. Students master core programming principles—including control structures, functions, and data structures—while gaining hands-on experience in debugging and writing efficient code. The curriculum emphasizes practical application through laboratory experiments and self-learning activities, enabling students to develop simple programs and apply concepts to real-life problems.

Course Objectives

1	To equip students with the ability to solve complex problems by designing structured algorithms and visual flowcharts before implementation.
2	To provide a comprehensive understanding of Python syntax, including variables, data types, operators, and various control structures like selection and iteration.
3	To familiarize students with the manipulation of built-in data structures such as strings, lists, set, tuples, and dictionaries for efficient data management.
4	To introduce the principles of modularity through the use of built-in and user-defined functions, focusing on code reusability and scope management.
5	To foster practical programming skills through laboratory experiments and real-world problem-solving

Course Pre-requisite

Topic	Course code	Course Title	Semester
Basic understanding of computer terminologies		IT lab taught in secondary level school	Secondary Education

Course Outcome

CO	Course Outcome	Blooms Taxonomy Level
CO1	Acquire foundational logical reasoning skills by designing flowcharts and algorithms and develop Python programs that effectively incorporate variables, fundamental data types, and standard input/output operations.	Apply
CO2	Optimize program execution through the strategic use of control structures and loop control statements.	Apply
CO3	Implement effective data management strategies using strings, lists, tuples, sets, and dictionaries by applying operations and built-in methods to solve real-world programming problems	Apply
CO4	Design and develop modular programs using functions, modules, and packages with proper scope management and argument passing techniques	Apply

CO-PO mapping

COURSE ARTICULATION MATRIX											
Course Outcomes	PROGRAM OUTCOMES										
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	3	2	2	2	-	-	-	-	-	3
CO2	3	3	3	2	2	-	-	-	-	-	3
CO3	3	3	3	3	3	-	-	-	-	-	3
CO4	3	2	3	3	3	-	-	-	-	-	3
Total	12	11	11	10	10	0	0	0	0	0	12
Correlation Level	3	2.75	2.75	2.5	2.5	0	0	0	0	0	3

Teaching and Assessment Scheme

Teaching Scheme/Week				Self-learning (SS)/Week	Total Hours/Semester	Credits C	Examination Scheme						
							Theory			Practical			Total
L	T	P	S				CIE	ESE	Total	CIE	ESE	Total	
3	0	3	0	6	90	4.5	20	60	80	20	40	60	140

L: Lecture (One unit is of one-hour duration with one credit), T: Tutorial (One unit is of one-hour duration with one credit), P: Practical (One unit is of one-hour duration with 0.5 credit), S: Project (One unit is of one-hour duration with 0.5 credit) SS: Self-Learning, CIE: Continuous Internal Evaluation, ESE: End Semester Examination

Syllabus - Major Topics

Module	Title	Major Topics	Instructional Hours	
			L	P
Module I	Problem Solving & Python Fundamentals	Theory Topics: Introduction to problem solving, problem-solving methods (algorithmic and heuristic), program development cycle, problem analysis, algorithm design, flowchart symbols and conventions, introduction to Python, features and applications of Python, Python versions, installation of Python and IDEs, overview of Python interpreter, variables, keywords, literals, built-in data types, type conversion, basic operators, input and output statements.	12	0
Module I	Problem Solving & Python Fundamentals	Laboratory experiments: • Introduction to computer laboratory rules, equipment usage, and safety measures. • Design Algorithm and flowchart for Simple Problems like- Simple Interest Calculation, Odd or Even Number Check • Installation of Python and IDEs • Write Python program for - Simple Interest Calculation, Arithmetic Operation, Degree Celsius to Fahrenheit Conversion	0	12
Module II	Control Structures & Looping Structure	Theory topics: Selection control structures (if, if-else, nested if, if-elif, match-case), iteration control structures (for loop, while loop) range() function, nested loops, loop control statements (break, continue, pass), problem solving using control structures.	12	0
Module II	Control Structures & Looping Structure	Laboratory experiments: • To design and implement Python programs that perform basic decision-making (Examples): Odd or Even, Leap Year, Simple Interest with field Validation • To design and implement Python programs that perform selection-based operations. (Examples): Display the day name based on the day number, Arithmetic Operation with choice • To design and implement Python programs that use loops to perform repetitive tasks: Multiplication Table, Pattern Printing	0	12
Module III	Strings & Data Structures -List, Tuple, Set and Dictionary	Theory topics: Strings and string operations, string indexing and slicing, common string methods, lists and list operations, list slicing and methods, tuples and their characteristics, sets and set operations, dictionaries and dictionary operations, real-life applications of Python data structures.	12	0
Module III	Strings & Data Structures -List, Tuple, Set and Dictionary	Laboratory experiments: • To design and implement Python programs that perform basic string manipulations such as splitting, counting the number of characters, converting a string to uppercase or lowercase, and reversing a string, demonstrating the use of string methods • To design and implement Python programs that demonstrate the use of lists and their operations, such as inserting, appending, deleting, sorting, slicing, and illustrating the use of list methods • To design and implement Python programs that demonstrate the use of dictionaries and their operations, including inserting, updating, deleting elements, displaying the entire dictionary, and showing only keys or values, illustrating the use of dictionary methods. • To design and implement Python programs that demonstrate the use of sets and their characteristics, including creating sets, display set elements, performing set operations such as union(), intersection(), and difference()	0	12
Module IV	Functional and Modular Programming in Python	Theory topics: Introduction to functions and advantages, Built-in functions (len(), min(), max(), sum()), User-defined functions (defining, calling, return values), Function arguments (Positional, Default), Variable scope (Local and Global), Module creation and usage, Packages and importing packages, Introduction to Numpy package	9	0

Module IV	Functional and Modular Programming in Python	Laboratory experiments: • To design and implement Python programs that demonstrate the use of built-in functions for performing common operations. • To design and implement Python programs that demonstrate the use of functions, including parameter passing and different types of arguments • To design and implement Python programs that demonstrate the creation and use of custom modules.	0	9
-----------	--	--	---	---

Detailed Syllabus

Subtopic	Student Learning Outcome	Mode of delivery (L/T)	Mapped CO	Taxonomy Level	Instructional Hours
Module I					
Introduction to Problem Solving: Problem definition, Problem-solving methods	Explain problem-solving approaches and distinguish between algorithmic and heuristic methods	L	CO1	Understand	1
Program Development Cycle	Explain the systematic program development process and its phases: Problem analysis, Design, Coding, Testing and Documentation	L	CO1	Understand	2
Introduction to computer laboratory rules, equipment usage, and safety measures.	Explain laboratory rules, use computer equipment properly, and follow safety measures.	P	CO1	Understand	3
Algorithm and Flowchart: Algorithm steps	Explain the concept and purpose of an algorithm in problem solving	L	CO1	Understand	1
Algorithm and Flowchart: Flowchart symbols and their meanings	Explain the basic flowchart symbols and their meanings.	L	CO1	Understand	1
Introduction to Python: Language versions, Features, Real-world applications	Identify Python ,Python Versions, features and its applications in real-world scenarios	L	CO1	Understand	1
Design Algorithm and flowchart of Simple Problems like- (Simple Interest Calculation/ Odd or Even Number)	Design step-by-step algorithms and draw appropriate flowcharts for simple computational problems	P	CO1	Apply	3
Python Installation: IDE installation (IDLE, PyCharm, VS Code), Python Interpreter	Install and configure Python and its development environment	L	CO1	Understand	1
Variables and Built-in Data Types: Literals, Keywords, Variable initialization	Explain variables, literals, keywords, and variable initialization in Python programming.	L	CO1	Understand	1
Variables and Built-in Data Types:Data types,Type() function ,Type Conversion	Explain appropriate built-in data types (Integer, Float, Boolean, String, None) to store different kinds of information in Python programs. Implement type conversion and type casting techniques.	L	CO1	Understand	1
Installation of Python and IDE(PyCharm/VSCode or others)	Install and configure Python and a suitable Integrated Development Environment (IDE) and also verify the installation, set up the development environment, and execute basic Python programs successfully	P	CO1	Apply	3
Operators in Python: Arithmetic, Assignment, Comparison, Logical, Identity, Membership, Bitwise operators	Apply various operators to perform computations and comparisons	L	CO1	Apply	2
Input and Output Statements: Comments, input() function, print() function, f-strings	Implement input/output operations for interactive programs	L	CO1	Apply	1
To design and implement simple Python programs that accept user input, perform basic mathematical computations, and display the results.	Apply fundamental Python concepts to develop and execute simple programs for basic real-world problems using appropriate variables, operators, expressions, and user input	P	CO1	Apply	3
Module II					
Selection Control Structure: Purpose, Types, Flowcharts	Explain the need and types of selection structure	L	CO2	Understand	1
if Statement: Syntax, Working, Flowchart, Real-life applications	Implement conditional logic using if statements	L	CO2	Apply	1

if-else Statement: Syntax, Working, Flowchart, Real-life applications	Develop programs with binary decision-making using if-else	L	CO2	Apply	1
To design and implement Python programs that perform basic decision-making	Apply Python programming concepts to develop and execute programs for decision-making with input validation, using variables, operators, expressions, and conditional statements.	P	CO2	Apply	3
Nested if and if-elif Statements: Syntax, Working, Flowchart	Implement multi-way decision structures using nested conditions	L	CO2	Apply	1
match-case Statement: Syntax, Working, Flowchart, Applications	Implement multi-way decision structure using match-case.	L	CO2	Understand	1
Iteration Control Structure: Purpose, Types, Flowcharts	Explain the concept and necessity of loops in programming	L	CO2	Understand	1
To design and implement Python programs that perform selection-based operations.	Apply Python programming concepts to develop and execute programs that perform selection-based tasks	P	CO2	Apply	3
For Loop: Syntax, Working, Flowchart, Real-life applications	Use for loop logic to implement repetitive tasks.	L	CO2	Apply	1
range() Function: Syntax, Working, negative step value	Apply range() function for controlled iteration	L	CO2	Apply	1
Nested Loops: Usage, Working, Real-life applications	Develop programs using nested iteration structures and pattern printing	L	CO2	Apply	1
To design and implement Python programs that use loops to perform repetitive tasks (for Loop)	Develop and execute Python programs using for loops to apply iteration, variables, and formatted output for tasks	P	CO2	Apply	3
While Loop: Syntax, Working, Flowchart, Real-life applications	Implement indefinite iteration using while loops	L	CO2	Apply	1
Loop Control Statements: break, continue, pass with applications	Understand loop control statements used for flow manipulation.	L	CO2	Understand	2
First Series Test -Practical	Develop and execute Python programs using variables, operators, input/output, conditional statements, and looping structures to solve basic computational problems and produce correct program outputs	P	CO2	Apply	3
Module III					
Strings and String Operations: Characteristics, Accessing, Operations (Concatenation, Repetition, Length, Membership)	Apply string operations in Python such as accessing, concatenation, repetition, length checking, and membership testing to solve programming problems.	L	CO3	Apply	1.5
String Indexing and Slicing: String Methods (upper, lower, split,strip,count,)	Apply methods to manipulate textual data through string slicing and built-in string methods in Python programs.	L	CO3	Apply	1.5
To design and implement Python programs that perform basic string manipulations such as splitting, counting the number of characters, converting a string to uppercase or lowercase, and reversing a string, demonstrating the use of string methods	Develop and execute Python programs that apply string manipulation techniques to split strings, count characters, change case, and reverse strings, reinforcing their understanding of string operations, methods, and program logic	P	CO3	Apply	3
Lists: Characteristics, Accessing elements, Operations (Concatenation, Repetition, Membership)	Utilize lists for storing and managing sequential data Apply list operations such as element access, concatenation, repetition, and membership testing in Python programs.	L	CO3	Apply	1.5
Lists: List Methods (append, pop, remove, insert, count, sort)	Apply list methods such as append, pop, remove, insert, count, and sort to manipulate and manage sequential data in Python programs	L	CO3	Apply	1.5
To design and implement Python programs that demonstrate the use of lists and their operations, such as inserting, appending, deleting, sorting, slicing, and illustrating the use of list methods	Develop and execute Python programs that apply list operations such as insertion, appending, deletion, sorting, displaying elements, and slicing	P	CO3	Apply	3

Dictionaries: Characteristics, Accessing elements, Operations (Adding, Updating, Removing, Traversing)	Apply dictionary operations such as adding, updating, removing, and traversing elements, using Python programs.	L	CO3	Apply	2
Dictionary Methods: get(), keys(), values(), items()	Apply dictionary methods to access, iterate, and manipulate key-value pairs.	L	CO3	Apply	1
To design and implement Python programs that demonstrate the use of dictionaries and their operations, including inserting, updating, deleting elements, displaying the entire dictionary, and showing only keys or values, illustrating the use of dictionary methods, key-value pairs	Develop and execute Python programs that apply dictionary operations such as insertion, updating, deletion, and displaying keys, values, and items	P	CO3	Apply	3
Tuples: Characteristics, Accessing elements, Methods (count(), index(), len(), max(), min())	Apply the tuple data structure to manage immutable sequential data	L	CO3	Understand	1.5
Sets: Characteristics, Accessing elements, Operations (union(), intersection(), difference())	Implement set operations for unique data management	L	CO3	Understand	1.5
To design and implement Python programs that demonstrate the use of sets and their characteristics, including creating sets, accessing elements, performing set operations such as union(), intersection(), and difference()	Develop and execute Python programs that apply set operations to manage collections of unique elements and to identify common, different, and combined elements from multiple data sets.	P	CO3	Apply	3
Module IV					
Introduction to Functions: Use, Advantages, Built-in functions (len(), min(), max(), sum())	Describe the purpose and benefits of modular programming	L	CO4	Understand	1
User-defined Functions: Defining, Calling, Return values	Develop modular programs using user-defined functions	L	CO4	Apply	2
To design and implement Python programs that demonstrates the use of built-in functions for performing common operations.	Develop and execute Python programs that apply built-in functions such as len(), max(), min(), and sum(), to process lists or strings, reinforcing their understanding of Python's inbuilt capabilities for sequence manipulation.	P	CO4	Apply	3
Function Arguments: Positional, Default Arguments	Implement functions with arguments (Positional and Default)	L	CO4	Apply	2
Scope of Variables: Local and Global scope	Understand and manage variable scope in programs	L	CO4	Understand	1
To design and implement Python programs that demonstrate the use of functions, including parameter passing and different types of arguments	Develop and execute Python programs that apply functions with parameters and default arguments to understanding of modular programming, code reuse, and function design in Python.	P	CO4	Apply	3
Module in Python: Purpose, Creating modules, Using modules, Built-in modules, Package in Python: Definition, importing packages	Develop reusable code modules and Organize related modules into packages	L	CO4	Understand	2
NumPy Array: Use, Importing numpy module, Create 1D Array using numpy, Attributes (shape, dtype)	Use the concept of NumPy arrays in mathematical and statistical computations.	L	CO4	Understand	1
Second Series Test -Practical	Develop Python programs using strings, data structures, and functions to perform data manipulation and solve basic programming problems	P	CO4	Apply	3

Suggested Student Activities for Self-Learning

Week	Self-Learning description	Hours
Module I		
1	- Identify real-life problems that can be solved using algorithmic or heuristic methods - Create a visual chart of the Program Development Cycle	3+3

2	• Draw different flowchart symbols and their use • Students design simple algorithms and flowcharts for daily life problems such as calculating total marks / finding the largest of two numbers/Simple formula solving	3+3
3	• Least five features of Python 2.7 and Python 3 • Compare Python IDLE, PyCharm, and VS Code features • Prepare a short comparison chart covering at the variable declaration rules in Python with a suitable example	2+2+2
4	• List the different operators in Python and find out their precedence. • Write Python programs in online Python Compilers(onlineGDB/Programiz/Nextleap etc) ◦ Write simple programs using variables and operators (simple interest, temperature conversion). ◦ Document common errors and their solutions.	2+4
Module II		
5	• Prepare a self-study sheet containing definitions, flowcharts, syntax, and real-life examples of if and if- else statements • ● Write Python programs in online Python Compilers (onlineGDB/Programiz/Nextleap etc) ◦ Write simple programs using if -else statements (Odd or Even/Leap Year etc).	3+3
6	• Draw a flowchart of match-case • Compare the match-case statement with the if-elif ladder by analyzing their advantages and limitations through examples • Identify and list problems suitable for solving with the match-case structure.	1+3+2
7	• Prepare a self-study sheet containing definitions ,syntax, usage and application • ● Write Python programs for printing various patterns using the loop and range() function. <pre> * ** *** **** 1 22 333 4444 1 12 123 1234 </pre>	3+3
8	• Explain the situations in which break, continue, and pass statements are used, with examples. • Prepare a short comparison chart of For Loop and While Loop. • Identify and debug infinite loop situations and document methods to prevent them	2+2+2
Module III		
9	• Create documentation outlining common string methods and their usage with examples • Perform string slicing operations with positive and negative indexing	3+3
10	• ● Prepare a Tutorial Video Script or Video about Python List (5 minutes) • Document the different operations on lists along with relevant built-in functions.	3+3
11	• Prepare a self-study sheet containing usage, operations and built-in methods of Dictionary • Create a Python program to manage student records using a dictionary. Your program should: ◦ Create a dictionary containing at least 5 students and their marks. ◦ Access the marks of a specific student, and safely handle the case where the student does not exist. ◦ Add a new student to the dictionary. ◦ Update the marks of an existing student	3+3
12	• Create comparison tables of : List vs Tuple vs Set • Create a Python program to ◦ Create two sets with some common and some unique elements ◦ Perform operations: ■ Union ■ Intersection ■ Difference	3+3
Module IV		
13	• Explore Python libraries used for Image Processing, Data Analysis, Visualization, and Machine Learning. • List at least 5 benefits of modular programming	3+3
14	• Write user-defined functions in Python that use parameters and return values [Example Like] ◦ Factorial of a Number ◦ Celsius to Fahrenheit Converter • Explore and differentiate between positional, default arguments in Python functions, supported by examples	3+3
15	• Prepare multiple-choice questions (MCQs) to enhance learning and practice. • Prepare 20 MCQs on topics related to Python programming (e.g., loops, strings, lists, dictionaries, tuples, sets, etc). • Containing five questions for each Course Outcome (CO) • Provide four options for each question. • Indicate the correct answer clearly for each question	6
Total Self-Learning hours		90

Learning Resources

TEXT BOOK			
Sl. No.	Title	Author	Publication
1	Introduction to Python Programming	Gowrishankar S, Veena A	CRC Press /Taylor & Francis
2	Let us Python	Yashavant Kanetkar, Aditya Kanetkar	BPB Publication

REFERENCE			
Sl. No.	Title	Author	Publication
1	Python 3 for Absolute Beginners	Tim Hall & J P Stacey	Apress
2	Python for Everyone	Cay S. Horstmann, Rance D. Necaise	Wiley
3	Prelude to Programming : Concepts & Design	Venit, S & Drake E	Addison Wesley (Pearson)

WEB RESOURCES		
Sl. No.	URL	Modules Covered
1	https://www.w3schools.com/python/	All
2	https://developers.google.com/edu/python	All
3	https://www.youtube.com/watch?v=8DvywoWv6fI	All
4	https://nptel.ac.in/courses/106106182	All

Major Equipment/Instruments/Components required for a batch of 30 students

Sl. No.	Particulars	Specifications	Quantity
1	Personal Computer	Intel Core i5/AMD Ryzen 5 (12th Gen or higher), 16GB RAM, 512GB SSD, 18.5" LED Monitor	30
2	Internet	Dedicated Leased Line or Broadband with minimum 100 Mbps speed and Wi-Fi 6 Support	1

Major Software required for a batch of 30 students

Sl. No.	Particulars	Specifications	Quantity
1	Operating System	Windows 10/11 Professional or Latest Ubuntu LTS.	30
2	Programming Language	Python 3.0 or more	30
3	Integrated Development Environment	VS Code/IDLE	30

Assessment Methodology - Practicum

Mode	Attendance	CA1	CE	CA2	CA3	CA4	CA5	ESE	
		Average of Self learning activities from each module	Continuous evaluation	Practical test (first half of experiments)	Practical test (second half of experiments)	Series Exam (2 modules)	Series Exam (2 modules)	Written Exam (theory)	Lab Examination (internal)
Duration				3 hours	3 hours	2 hours	2 hours	2.5 hours	3 hours
Exam mark		10	50	40	40	50	50	60	40
Converted to			10	10	10	10	10		
Marks	5	5	10	10		10		60	40
Total	40							100	
Tentative schedule	End of semester	By 4th week	By 7th week	By 11th week	By 14th week	8th week	15th week	End of semester	

Scheme of Evaluation - Practical (Continuous Internal Evaluation)

Sl. No.	Criteria	Description	Marks
1	Preparation & Punctuality	Attendance, timely submission of record, pre-lab preparation, understanding of experiment objectives and theory.	10
2	Experimental Setup & Procedure	Ability to identify components/equipment, correct circuit connections/setup, systematic and safe execution of the procedure.	10
3	Observation & Data Recording	Accuracy and neatness of readings, correct use of units, tabulation, and systematic recording of results.	5
4	Analysis & Result Interpretation	Proper calculations, graphical representation, result verification, discussion, and logical conclusion.	10
5	Viva Voce / Concept Understanding	Clarity of theoretical concepts, explanation of procedure, ability to answer questions related to the experiment.	10
6	Workmanship, Discipline & Teamwork	Laboratory discipline, care of instruments, teamwork, attitude, and safety practices.	5
Total			50

Scheme of Evaluation - Practical Test

Category	Things to evaluate	Marks
Write-up / Procedure (before experiment)	Aim, circuit diagram/flowchart, stepwise procedure.	10
Experiment Setup & Execution	Correct circuit connections / coding / setup. Handling of instruments/software. Accuracy of execution.	10
Observation & Result / Output	Proper tabulation of readings. Calculations/graphs/program output. Correctness of result / expected outcome.	8
Viva Voce	Conceptual understanding of experiment. Related theoretical knowledge.	8
Record	Completion & neatness of practical record. Proper certification by faculty.	4
Total		40

Series Examination Pattern- Theory

Time	Module (Series 1/ Series 2)	Part A	Marks	Part B	Marks	Part C (With choice)	Marks	Total
2 hours	Any one module	2	1	3	3	2	7	25
	Any one module	2	1	3	3	2	7	25
	Number	4		6		4		50

End Semester Examination Pattern of Question Paper - Theory

Duration	Module	TYPE OF QUESTIONS						
		Part A (2 Questions from each module)		Part B 2 out of 3 questions from each module)		Part C (1 set questions with choice from each module)		Total Marks
		No. of Questions	Marks (1 mark each)	No. of Questions	Marks (3 marks each)	No. of Questions	Marks (7 marks each)	
2.5 hours	1	2	2	3	6	1 set	7	15
	2	2	2	3	6	1 set	7	15
	3	2	2	3	6	1 set	7	15
	4	2	2	3	6	1 set	7	15
		8	8	12	24	4 sets	28	60